

Model Denial of Service (DoS)

1. Understanding the Model DoS Threat

Denial of Service attacks against AI systems are fundamentally different from classic network DoS attacks, yet they share the same objective: make the target unavailable. In traditional DoS, attackers flood a network pipe or exhaust server connections. Against AI systems, the primary lever is computational cost, not bandwidth. A single carefully constructed input can consume hundreds or thousands of times more resources than a benign input of the same byte length.

This asymmetry is the defining characteristic of model DoS. The attacker's cost to craft a destructive input may be trivial — a few seconds of experimentation with the API — while the defender's cost to process that input may be catastrophic. For security AI specifically, the strategic value to an attacker is obvious: if the system that detects intrusions or analyses malware is unavailable, the attacker operates in darkness. Unlike data exfiltration or model compromise, DoS does not require defeating the model's logic — it simply requires preventing it from running.

AI model DoS spans a spectrum of sophistication: from crude request flooding (no AI knowledge required) to precisely engineered sponge examples (requires understanding of model architecture and inference dynamics). Defenders must address both ends of this spectrum.

Asymmetry Principle: In model DoS, the attacker's cost to generate a destructive input is far lower than the defender's cost to process it. A single sponge example may cost milliseconds to craft but minutes to process, multiplied across concurrent requests.

2. Taxonomy of Model DoS Attack Vectors

Attack Vector	Mechanism	Resource Targeted	Sophistication
Request flooding	High volume of normal requests overwhelms capacity	Throughput / queuing	Low — no AI knowledge required
Maximum-size inputs	Repeated submissions at allowed size limits	Compute + memory	Low — exploits permitted limits
Sponge examples	Inputs crafted to trigger worst-case computation paths	CPU / GPU / time	High — requires model knowledge
Algorithmic complexity inputs	Inputs that hit worst-case algorithm complexity (e.g., $O(n^2)$ attention)	CPU + memory	Medium-High — requires attention/architecture knowledge
GPU memory exhaustion	Inputs maximising GPU memory allocation per inference	GPU VRAM	Medium — requires knowledge of batch/sequence effects

Adversarial DoS inputs	Adversarial examples crafted for computational cost rather than misclassification	CPU / GPU	High — adversarial optimisation with DoS objective
Cascade requests	Inputs that cause AI to trigger expensive downstream API chains	Compute + downstream services	Medium — exploits AI agent architecture

3. Sponge Examples: A Deep Dive

Sponge examples were formally studied by security researchers as inputs designed to maximally absorb the computational resources of machine learning inference. The term captures their key property: like a sponge, they absorb far more than their apparent size would suggest.

For transformer-based language models, the primary sponge mechanism exploits the self-attention mechanism. Self-attention has quadratic complexity with respect to sequence length — doubling the sequence length quadruples the computation. A sponge example crafted to maximally utilise the model's context window therefore imposes quadratic cost, while a short input imposes minimal cost. The byte length ratio between attack and benign inputs may be 10:1; the compute ratio may be 100:1 or greater.

For computer vision models, sponge examples exploit detection architectures that generate proposals proportional to image complexity. An adversarially constructed image maximising the number of object proposals (bounding boxes) causes the non-maximum suppression step to process far more candidates than normal, consuming disproportionate compute. The image itself may appear as visual noise to a human observer.

What makes sponge examples particularly dangerous is that they are not obviously malicious to signature-based filters. They do not contain injection payloads or known attack strings — they are inputs that happen to be computationally catastrophic for the specific model architecture.

Real-World Research — Sponge Examples Against NLP and Vision Models (Shumailov et al., 2021):

Researchers from the University of Cambridge and the University of Edinburgh formally introduced sponge examples, demonstrating that inputs could be crafted to increase energy consumption and latency of inference by factors of 10 to 100 across multiple model types including BERT-based NLP models and computer vision classifiers. The research highlighted that standard defences (input size limits, rate limiting) were insufficient when attackers could craft within-limit inputs that triggered worst-case computation. This established sponge examples as a legitimate adversarial threat category distinct from misclassification-focused adversarial examples.

4. Algorithmic Complexity Attacks on AI Systems

Many AI algorithms have average-case performance far better than their worst-case performance. Attackers who understand these gaps can craft inputs that force worst-case execution, achieving disproportionate resource exhaustion without exceeding any input size or rate limit.

For transformer attention mechanisms, the standard $O(n^2)$ complexity with sequence length is the most exploitable gap. A model with a 32,000 token context window may process typical 500-token

queries in milliseconds, but a 32,000-token query with structure designed to maximise cross-token attention interactions can require thousands of times more compute. The attacker sends within-limit inputs that are disproportionately expensive.

Beyond attention, other algorithmic complexity vulnerabilities include: tree-based inference models with inputs crafted to maximise tree depth traversal; inference-time search algorithms in planning AI that can be steered toward exponentially deep search trees; and iterative refinement models where adversarial inputs prevent convergence, causing maximum-iteration execution. Each requires domain knowledge of the specific model architecture, making these attacks targeted rather than opportunistic.

5. GPU Memory Exhaustion

Modern AI inference depends heavily on GPU acceleration. GPU VRAM is a finite, non-swappable resource — unlike system RAM, GPU memory cannot be paged to disk. When GPU memory is exhausted, inference either fails with an out-of-memory error or falls back to dramatically slower CPU processing.

Attackers target GPU memory by submitting inputs that maximise the memory footprint per inference: maximum sequence lengths, maximum batch sizes, maximum image resolutions, or structured inputs that cause large intermediate activation tensors. A GPU with 40GB VRAM might handle 100 normal concurrent inference requests but fail on 5 specially crafted maximum-memory requests submitted simultaneously.

The operational impact for security AI using GPU-accelerated inference (such as malware analysis models or network traffic classifiers) is availability degradation precisely when the system may be under concurrent attack. An adversary who has discovered how to trigger GPU exhaustion can disable the security AI during the opening phase of a larger intrusion campaign.

6. Request Flooding: Simple but Effective

At the simplest end of the DoS spectrum, request flooding requires no understanding of AI internals whatsoever. The attacker submits the maximum permitted volume of requests — each individually legitimate — at a rate that exhausts the service's capacity. Queues back up, latency rises, timeouts occur, and real requests are delayed or dropped.

For security AI, the consequence of request flooding is analytical backlog. An intrusion detection AI receiving 10,000 alerts per second but capable of processing 1,000 per second accumulates a backlog. Actual threats buried in that backlog experience analysis delays measured in hours rather than milliseconds. By the time the AI surfaces the alert, the attacker has completed their objective.

Request flooding is the easiest DoS vector to detect (traffic volume anomalies are conspicuous) but also the easiest for well-resourced attackers to scale using botnets, making rate limiting alone insufficient for availability guarantees.

7. Detection: Identifying Model DoS in Progress

Detection of model DoS requires instrumentation at the inference layer, not just at the network layer. Standard network-level DDoS detection misses compute-layer exhaustion caused by sponge examples or algorithmic complexity attacks, because the attack traffic may be well within normal volume thresholds.

- **Per-request resource tracking:** Instrument each inference call to record CPU time, GPU time, memory allocation, and wall-clock latency. Establish baselines for each request type. Alert on requests that consume significantly more resources than the baseline for their type. A 100ms average request taking 10 seconds is a DoS indicator regardless of the traffic volume.
- **Queue depth monitoring:** Monitor inference queue depth and time-in-queue metrics. Sustained queue growth indicates the system is consuming requests more slowly than it is receiving them — a DoS in progress.
- **Request pattern analysis:** Detect statistical anomalies in request characteristics: systematic maximum-length submissions, high variance in processing time from a single source, or uniform submission of inputs triggering slow-path processing. Pattern analysis distinguishes legitimate heavy users from targeted DoS.
- **GPU utilisation telemetry:** Monitor GPU memory usage per inference. Sustained near-capacity GPU memory utilisation from a small number of request sources indicates GPU memory exhaustion attacks.

8. Prevention: Input Limits and Timeout Controls

Input limits are the most direct defence against sponge examples and size-based DoS. Enforcing strict maximums on sequence length, image resolution, batch size, and other input dimensions caps the per-request computational cost, preventing worst-case algorithmic complexity exploitation. The challenge is calibrating limits to be restrictive enough to prevent abuse while permissive enough for legitimate use cases.

Timeout controls provide a last-resort defence. Any inference call that exceeds a defined wall-clock time limit is terminated and returns an error. Timeouts prevent any single request from consuming resources indefinitely, regardless of whether it is a sponge example, a complexity attack, or a simple slow request caused by infrastructure issues. The timeout threshold must be set above the 99th percentile of legitimate request latency to avoid disrupting normal users while still catching pathological cases.

- Set input size limits (sequence length, image resolution, batch size) at values that cover legitimate use cases but exclude worst-case algorithmic complexity triggers.
- Enforce per-request CPU and GPU time budgets. Terminate requests exceeding budget rather than waiting for natural completion.
- Apply wall-clock timeouts independently of compute-time budgets — network delays and queuing must not allow budget evasion.
- Use exponential backoff and circuit breakers for downstream calls triggered by AI outputs, preventing cascade DoS from AI agent architectures.

9. Prevention: Rate Limiting, Auto-Scaling, and Load Balancing

Rate limiting caps the number of requests a single client (identified by API key, IP address, or authenticated identity) can submit per unit time. It is the primary defence against request flooding and

maximum-size input attacks from a single source. Effective rate limits are set per authenticated identity rather than per IP address, as IP-based limits are easily evaded by distributed botnet attacks.

Auto-scaling dynamically provisions additional inference capacity when load increases. Cloud-based AI inference services can scale horizontally — spinning up additional GPU instances within seconds — to absorb legitimate traffic spikes and to dilute DoS attacks that stay within rate limits. Auto-scaling is not free (additional capacity has cost), and it is most effective when combined with rate limiting that prevents attackers from causing unbounded scaling costs.

Load balancing distributes requests across inference instances, preventing any single instance from becoming a bottleneck. For expensive requests that slip through input limits, load balancing ensures that a single slow request does not block all users served by that instance.

10. Availability as a Security Property for AI Systems

In the CIA triad (Confidentiality, Integrity, Availability), availability is sometimes treated as the least critical property for security systems. Model DoS attacks demonstrate that for security AI, availability is arguably the most critical: a security AI that is unavailable cannot perform any of its confidentiality or integrity protection functions.

Security teams must apply SLA thinking to AI-based security tools just as they do to human analysts: what is the acceptable response latency for a threat alert? What is the acceptable queue backlog before escalation? What is the recovery time objective if an AI service becomes unavailable? Answers to these questions drive the architecture of DoS defences — whether rate limiting alone is sufficient, or whether dedicated hardware, failover instances, and circuit breakers are required.

The analogy that grounds this: an airport that has a perfect baggage screening protocol but shuts down the screening lanes when passenger volume spikes provides no security during peak hours — precisely when security risk is highest. Availability under load is not a nice-to-have for security AI; it is a foundational requirement.

Key Terms Glossary

Term	Definition
Model Denial of Service (DoS)	An attack that makes an AI system unavailable or unusably slow by exhausting its computational resources (compute, memory, GPU, or time) rather than by exploiting a logic vulnerability.
Sponge example	An input crafted to maximally absorb AI inference resources — disproportionately expensive to process relative to its size — without necessarily causing misclassification.
Algorithmic complexity attack	An attack that exploits worst-case algorithm performance (e.g., $O(n^2)$ attention scaling) to cause resource exhaustion with within-limit inputs.
Request flooding	A DoS technique that submits a high volume of individually legitimate requests, overwhelming AI service capacity through sheer quantity.
GPU memory exhaustion	A DoS technique that submits inputs maximising GPU VRAM

	allocation per inference, causing out-of-memory failures or forced fallback to slower CPU processing.
Adversarial DoS input	An adversarially optimised input crafted to maximise computational cost of inference rather than to cause misclassification — combining adversarial example methodology with a DoS objective.
Per-request resource tracking	Instrumentation that measures CPU time, GPU time, memory, and latency for each inference call, enabling detection of anomalously expensive requests indicative of DoS.
Input limits	Strict maximum values for input dimensions (sequence length, image resolution, batch size) that cap per-request computational cost, preventing worst-case algorithm exploitation.
Timeout controls	Hard limits on inference wall-clock time; requests exceeding the threshold are terminated and return an error, preventing resource monopolisation by any single request.
Rate limiting	Caps on requests per unit time per authenticated client, defending against request flooding and maximum-size input attacks from single sources.
Auto-scaling	Dynamic provisioning of additional inference capacity in response to load increases, absorbing legitimate traffic spikes and diluting DoS attacks within rate limits.
CIA triad (Availability)	In AI security contexts, availability means the AI system must be able to process requests within acceptable latency under adversarial load conditions — a primary security property for security AI.

Quick Revision Checklist

- Model DoS makes AI unavailable by exhausting resources — compute, memory, GPU, or time — rather than by compromising model logic.
- The defining characteristic is asymmetry: attacker cost to craft a destructive input is far lower than defender cost to process it.
- Know the six attack vectors: request flooding, maximum-size inputs, sponge examples, algorithmic complexity attacks, GPU memory exhaustion, and adversarial DoS inputs.
- Sponge examples exploit worst-case algorithm performance (e.g., $O(n^2)$ transformer attention) and look like legitimate inputs to signature-based filters.
- GPU memory is a finite non-swappable resource — exhausting it causes inference failure or CPU fallback, not graceful degradation.
- Detection requires per-request resource tracking and queue monitoring at the inference layer, not just network-level DDoS detection.
- Primary prevention controls: input size limits, per-request compute/time budgets, and wall-clock timeouts.
- Secondary controls: rate limiting per authenticated identity, auto-scaling for capacity, and load balancing across instances.

- For security AI, availability is the most critical CIA property — an unavailable security AI cannot protect anything.
- Exam tip: sponge examples are a high-value topic unique to AI security — understand their mechanism, their distinction from misclassification-focused adversarial examples, and the defences that specifically address them.

10 Exam-Based MCQs — Model Denial of Service

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. *Scenario:* A threat intelligence platform runs a large language model to summarise threat reports. During a load test, the security team discovers that submitting requests at the maximum allowed token length (16,000 tokens) causes inference latency to increase by a factor of 50 compared to typical 200-token requests. The platform processes requests from authenticated external partners with no per-request compute budgets enforced. Which control should the security team implement FIRST to address the identified vulnerability?

- A) Encrypt all inference API traffic to prevent attackers from crafting targeted inputs
- B) Enforce strict maximum sequence length limits on all API requests
- C) Retrain the language model to process long inputs more efficiently
- D) Increase GPU allocation to handle maximum-length requests without degradation

Answer: B **Explanation:** B is correct because enforcing a maximum sequence length directly caps the per-request computational cost, preventing attackers from exploiting quadratic attention scaling. A is incorrect — encryption prevents eavesdropping but does not prevent an attacker who controls the request from sending long sequences. C is incorrect — retraining for efficiency reduces cost per token but does not eliminate the $O(n^2)$ scaling problem; attackers can still craft maximum-allowed inputs. D is incorrect — increasing GPU capacity enables the attacker to scale their attack; it does not remove the asymmetry.

Q2. Which of the following BEST describes the core characteristic of a sponge example?

- A) An input that causes the AI model to produce a wildly incorrect output
- B) An input that appears normal in size but consumes disproportionately large computational resources during inference
- C) An input that poisons the AI model's training data to degrade future performance
- D) An input that causes the AI model to leak its training data through its output

Answer: B **Explanation:** B is correct — the defining property of a sponge example is the asymmetry between apparent size and processing cost. A describes a misclassification adversarial example, which has a different objective. C describes training data poisoning, which occurs during model training. D describes a training data extraction or memorisation attack.

Q3. *Scenario:* A security operations centre observes that its AI-powered malware analysis sandbox is receiving a sustained stream of file submissions from multiple external IP addresses. Each submitted file is

exactly at the maximum allowed size and causes analysis times 40 times longer than typical samples. The SOC has not identified any known malware in the submissions. Simultaneously, a low-and-slow intrusion attempt has been detected in a different part of the network. What is the MOST likely intent behind this attack pattern?

- A) Exfiltrating the model's training data through response timing
- B) Causing the malware analysis AI to be unavailable during the early phase of a breach
- C) Manipulating the malware analysis AI into misclassifying the payload as benign
- D) Extracting the model's decision boundaries for evasion research

Answer: B **Explanation:** B is correct — the pattern (flooding with maximally expensive inputs to exhaust resources) is a DoS objective, not a confidentiality or integrity attack. Disabling the security AI during the opening phase of an intrusion is the strategic motivation. A describes a timing side-channel attack. C describes a model evasion attack (crafted to avoid detection, not to exhaust resources). D describes model extraction.

Q4. Which of the following correctly explains WHY transformer-based language models are particularly vulnerable to algorithmic complexity DoS attacks?

- A) Transformers store training data verbatim and must search it linearly for each query
- B) Transformer self-attention has quadratic complexity with respect to input sequence length
- C) Transformers use GPU memory that cannot be shared between concurrent requests
- D) Transformer inference requires gradient computation, which scales with input depth

Answer: B **Explanation:** B is correct — the standard self-attention mechanism computes attention scores for every pair of tokens in the sequence, resulting in $O(n^2)$ complexity. Doubling sequence length quadruples compute. A is incorrect — transformers do not store training data verbatim; they encode parameters. C is incorrect — while GPU memory sharing is a consideration, it is not the specific mechanism of complexity-based DoS. D is incorrect — inference does not require gradient computation; gradients are used only during training.

Q5. What distinguishes model DoS from traditional network Denial of Service attacks?

- A) Model DoS requires a larger botnet to execute effectively
- B) Model DoS exploits computational cost asymmetry rather than network bandwidth exhaustion
- C) Model DoS permanently damages the model weights, requiring retraining
- D) Model DoS targets the model's confidentiality rather than its availability

Answer: B **Explanation:** B is correct — model DoS weaponises the gap between attacker cost (crafting the input) and defender cost (processing it). Traditional network DoS exhausts bandwidth or connections. A is incorrect — model DoS can be highly effective with a single client sending within-rate-limit expensive requests. C is incorrect — model DoS is an inference-time attack and does not modify model weights. D is incorrect — model DoS is an availability attack, not a confidentiality attack.

Q6. A security architect recommends setting a wall-clock timeout of 5 seconds on all AI inference requests. What is the PRIMARY security benefit of this control?

- A) It prevents attackers from intercepting AI outputs in transit
- B) It limits GPU memory allocation per request
- C) It prevents any single request from monopolising resources indefinitely, regardless of input type
- D) It enforces rate limits on high-volume request sources

Answer: C **Explanation:** C is correct — timeouts are the last-resort containment control. Regardless of whether a request is a sponge example, a complexity attack, or an infrastructure glitch, the timeout ensures the request cannot run forever. A is a transport security concern, not a timeout concern. B is incorrect — timeouts do not directly limit memory allocation; separate memory limits do that. D is incorrect — rate limits address request volume; timeouts address per-request duration.

Q7. Which detection approach is MOST effective at identifying sponge example attacks that stay within input size and rate limits?

- A) Network packet inspection for known attack signatures
- B) Per-request CPU and GPU time monitoring with baseline comparison
- C) Training data integrity verification
- D) Output scanning for injection payloads

Answer: B **Explanation:** B is correct — sponge examples look like legitimate inputs to network inspectors and size/rate monitors. The distinguishing characteristic is their disproportionate processing cost. Per-request resource monitoring with baseline comparison catches this signal. A (packet inspection) detects known attack patterns but cannot identify within-spec inputs causing worst-case computation. C (training data integrity) is a different concern. D (output scanning) addresses insecure output handling, not DoS.

Q8. An AI security service uses auto-scaling to add inference instances when load increases. What is the PRIMARY limitation of auto-scaling as a sole DoS defence?

- A) Auto-scaling is too slow to respond to flash DoS attacks
- B) Auto-scaling increases cost and can be weaponised to cause financial denial of service
- C) Auto-scaling does not work for GPU-based AI inference
- D) Auto-scaling permanently changes the model architecture

Answer: B **Explanation:** B is correct — auto-scaling absorbs load by provisioning paid capacity. An attacker who forces the service to scale to thousands of GPU instances can cause a financial DoS: the service remains technically available but becomes prohibitively expensive to operate. Rate limiting is required alongside auto-scaling to prevent this. A is partially true (scaling has a lag) but is not the PRIMARY limitation. C is incorrect — cloud GPU auto-scaling is widely available. D is incorrect — auto-scaling adds instances, not modifies model parameters.

Q9. Why is GPU memory exhaustion particularly serious for AI systems compared to RAM exhaustion?

- A) GPU memory is encrypted and cannot be accessed by other processes

- B) GPU memory cannot be paged to disk, so exhaustion causes immediate inference failure rather than gradual degradation
- C) GPU memory is shared across all users of a cloud service, amplifying the impact
- D) GPU memory exhaustion permanently corrupts the model weights stored there

Answer: B **Explanation:** B is correct — unlike system RAM, GPU VRAM cannot be transparently paged to disk (swapped). When it is exhausted, inference fails with an out-of-memory error or is forced to the dramatically slower CPU path. System RAM exhaustion causes gradual paging performance degradation; GPU exhaustion causes immediate failure. A is incorrect — GPU memory protection is not the key issue here. C is incorrect — GPU memory is typically isolated per customer in cloud environments. D is incorrect — model weights are not corrupted by memory exhaustion.

Q10. Which statement BEST describes why availability is especially critical for security AI compared to other AI application types?

- A) Security AI operates on more sensitive data than other AI types, making uptime a compliance requirement
- B) An unavailable security AI cannot perform any of its protection functions, so attackers specifically target it before or during an attack
- C) Security AI is more expensive per inference than other AI types, making DoS attacks more financially attractive
- D) Security AI uses larger models than other AI types, making it inherently more vulnerable to sponge examples

Answer: B **Explanation:** B is correct — security AI (intrusion detection, malware analysis, anomaly detection) is the last line of automated defence. If it is unavailable, threats proceed undetected. Attackers have strong strategic motivation to disable security AI before or during an intrusion campaign. A is partially true (compliance requires uptime) but does not capture the specific strategic reason availability matters for security AI. C is not a primary factor. D may be true but is a secondary concern compared to the strategic motivation.